33. Deadlocks, Races, and Cancellation Patterns

33. Deadlocks, Races, and Cancellation Patterns

Phase: Phase 5 — Concurrency
Estimated time: 1–2 days
Slug: `33-deadlocks-races`

Learning objectives

- Recognize classic deadlock patterns
- Eliminate data races systematically
- Cancel in-flight work with context
- Write concurrency regression tests

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Deadlocks

Common causes: - Lock ordering A then B vs B then A - Two goroutines waiting on each other unbuffered - WaitGroup Add inside the goroutine incorrectly

Races

A data race: concurrent access to a variable, at least one write, no sync. Fix by synchronizing or not sharing.

Cancellation pattern

```
func worker(ctx context.Context, in <-chan Task) error {
    for {
        select {
        case <-ctx.Done():
            return ctx.Err()
        case t, ok := <-in:
            if !ok { return nil }
            if err := handle(ctx, t); err != nil { return err }
        }
    }
}
```

Common mistakes (especially from Python instincts)

- “It didn’t race on my machine” without -race
- Fixing races by sleeping
- Nested locks without ordering rules

Exercises

Work in `exercises/33-deadlocks-races/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/33-deadlocks-races/` until you have an honest failing attempt.

Self-check

1. Define data race
2. Give a deadlock example
3. How context prevents abandoned work?

Further reading

- https://go.dev/doc/articles/race_detector